

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES**FAST SCHOOL OF COMPUTING**

Department of Computer Science

**Data Structures****Lab Manual**

Lab Instructor	Mr. Jawad Khan
Section	BCS-3H
Semester	Fall 2026

Lab 03: Advanced Singly Linked List Problems and Doubly Linked Lists

1. Learning Objectives

After completing this lab, students will be able to:

1. Extend a singly linked list ADT by implementing deletion at the head and tail.
2. Implement a destructor that correctly frees every remaining node, avoiding memory leaks.
3. Reverse a singly linked list iteratively, in place, using $O(1)$ extra space.
4. Merge two sorted singly linked lists into one sorted list by relinking existing nodes.
5. Understand the structure and trade-offs of a doubly linked list compared to a singly linked list.
6. Implement a doubly linked list ADT supporting insertion, deletion, and traversal in both directions.
7. Apply bidirectional traversal to check whether a doubly linked list reads the same forwards and backwards.
8. Delete a given node from a doubly linked list in $O(1)$ time using only a pointer to that node.

2. Lab Instructions

- Create a separate .cpp file for each task.
- Use meaningful variable, function, and class names.
- Properly indent and format your code.
- Use meaningful prompts and output labels.
- Add comments where the logic is not immediately obvious.
- Do not use global variables unless explicitly required.
- Do not use STL containers such as vector, list, or array unless explicitly permitted.
- Dynamically allocated memory must be properly released.
- Make sure there are no memory leaks or dangling pointers.
- Test your program with normal, boundary, and invalid inputs.
- For tasks containing a time-complexity constraint, the required complexity must be followed.

3. Theory and Concept Review

3.1 Recap: The Singly Linked List ADT

In the previous lab, you implemented a `LinkedList<T>` class template with a nested `Node` class, along with `insertAtHead`, `insertAtTail`, `display`, `size`, `isEmpty`, `findMiddle`, `findNthFromEnd`, and `hasCycle`. This lab builds directly on that class.

```
template <typename T>
class LinkedList {
private:
    class Node {
    public:
        T data;
        Node* next;

        Node(T value) {
            data = value;
            next = nullptr;
        }
    };

    Node* head;

public:
    LinkedList() { head = nullptr; }

    void insertAtHead(T value);
    void insertAtTail(T value);
    void display();
    int size();
    bool isEmpty();
    Node* findMiddle();
    Node* findNthFromEnd(int k);
    bool hasCycle();

    // You will add the missing pieces below in Task 1.
};
```

3.2 Deletion in a Singly Linked List

Removing a node means detaching it from the chain and freeing its memory. Deleting the first node is simple, since head can be moved directly to the second node before the old first node is freed. Deleting the last node is more involved: because a singly linked list only stores a next pointer, the node before the one being removed must be located first, so that its next pointer can be updated once the last node is gone. Without knowing that earlier node in advance, there would be no way to reconnect the list after removing the tail.

You are not given working code for either operation here. Designing deleteAtHead, deleteAtTail, and a destructor that frees every remaining node is your task in this lab.

3.3 What Is a Doubly Linked List?

A doubly linked list is a linear data structure similar to a singly linked list, except that every node stores two pointers instead of one: a next pointer to the following node, and a prev pointer back to the preceding node. This allows the list to be traversed in either direction, starting from any node.

```
template <typename T>
class DoublyLinkedList {
private:
    class Node {
    public:
        T data;
        Node* prev;
        Node* next;

        Node(T value) {
            data = value;
            prev = nullptr;
            next = nullptr;
        }
    };

    Node* head;

    // ... the rest of DoublyLinkedList is your task to design in Task 4
};
```

Maintaining two pointers per node costs extra memory compared to a singly linked list, but it brings genuine benefits beyond just backward traversal. Consider deleting the last node: in a singly linked list, the node before it has to be located by walking from the head and stopping one step early, since there is no other way to reach it. In a doubly linked list, once the last node itself has been reached, its own prev pointer already gives direct access to the node before it. No separate tracking of one step behind is required.

3.4 Working Inward from Both Ends

Because every node in a doubly linked list can be reached from either direction, a second pointer can start at the tail and move backward while a first pointer starts at the head and moves forward, the two working toward the middle simultaneously. In a singly linked list this is not possible without first finding the tail some other way, since there is no way to step backward from it. This idea becomes useful whenever a problem needs to compare or process elements from both ends of a list at the same time.

3.5 Deleting a Node You Already Have a Pointer To

Normally, deleting a specific node requires traversing from the head to find it, which costs $O(n)$. But if code elsewhere already holds a pointer directly to the node that needs to be removed, a doubly linked list can unlink it immediately: the node's own prev and next pointers already identify its neighbors, so those two neighbors can be reconnected to each other without any traversal at all. A singly linked list cannot do this in the same way, since a node has no way to reach the node before it.

4. Lab Tasks

Task 1: Complete the Singly Linked List ADT (Deletion)

Extend the `LinkedList<T>` class you built in the previous lab by adding deletion and proper cleanup. Do not use `vector`, `list`, or any other STL container.

Required Functions

Add the following to your existing class. Each row states only what the function must do; designing how it does it is part of the task.

<code>void deleteAtHead();</code>	Design a function that removes the node currently at the head of the list.
<code>void deleteAtTail();</code>	Design a function that removes the node currently at the tail of the list.
<code>~LinkedList();</code>	Design a destructor that deletes every remaining node so that no memory is leaked when the list is destroyed.

Complexity Requirement

- `deleteAtHead` must run in $O(1)$.
- `deleteAtTail` must run in $O(n)$ (no tail pointer for this task).
- The destructor must run in $O(n)$.

Required Behaviour

- Deleting from an empty list must not crash the program (it should simply do nothing, or print a message).
- Deleting the only remaining node must correctly reset head to `nullptr`.

Example

```
List: 10 20 30 40

deleteAtHead()
display()  ->  20 30 40

deleteAtTail()
display()  ->  20 30
```

Task 2: Reverse a Linked List (Iterative)

Write a function that reverses a singly linked list in place, so that the last node becomes the first and the first becomes the last.

```
void reverse();
```

Important Complexity Constraint

You are required to reverse the list iteratively, in place, with $O(n)$ time complexity and $O(1)$ extra space.

Restrictions

- Do not use recursion for this task.
- Do not copy values into an array, reverse the array, and rebuild the list.
- Do not use any STL container.
- You must update head to point to the new first node once the reversal is complete.
- Clearly state the time and space complexity of your solution in a comment.

Example

Before: 10 20 30 40

reverse()

After: 40 30 20 10

Task 3: Merge Two Sorted Linked Lists

Given two singly linked lists that are each already sorted in ascending order, merge them into a single sorted list. The merge must be performed by relinking the existing nodes from both lists; no new nodes may be created. After merging, the list that this function is called on should contain every node from both lists in sorted order, and the other list should be left empty.

```
void mergeSorted(LinkedList<T>& other);
```

Important Complexity Constraint

A solution that copies every value into a new array, sorts it, and rebuilds a list from scratch would work, but it ignores the fact that both lists are already sorted. You are required to produce the merged list by walking both lists once each and relinking their existing nodes directly.

Restrictions

- Do not allocate any new nodes; only relink the nodes that already exist in the two lists.
- Do not copy values into an array or any other auxiliary structure.
- Do not use any STL container.
- You may assume both lists are already sorted in ascending order before this function is called.
- Clearly state the time and space complexity of your solution in a comment.

Required Behaviour

- Calling this function when either list is empty must still produce the correct merged result.
- After the merge, the other list must be left empty (its head reset to nullptr), since its nodes now belong to this list.

Example

```

List A: 1 3 5
List B: 2 4 6

A.mergeSorted(B)

A: 1 2 3 4 5 6
B: (empty)

```

Task 4: Complete the Doubly Linked List ADT

Implement a class template named `DoublyLinkedList` that manages a doubly linked list of any data type, with the node implemented as a nested `Node` class holding data, prev, and next, as shown in Section 3.3. Do not use `vector`, `list`, or any other STL container.

Required Node Structure

```

class Node {
public:
    T data;
    Node* prev;
    Node* next;
};

```

Required Functions

Each row states only what the function must do; designing how it does it, including how the prev and next pointers of surrounding nodes are updated, is part of the task.

<code>void insertAtHead(T value);</code>	Design a function that inserts a new node containing value at the head of the list.
<code>void insertAtTail(T value);</code>	Design a function that inserts a new node containing value at the tail of the list.
<code>void deleteAtHead();</code>	Design a function that removes the node currently at the head of the list.
<code>void deleteAtTail();</code>	Design a function that removes the node currently at the tail of the list.
<code>void display();</code>	Design a function that prints every node's value, in order, from head to tail.
<code>void displayReverse();</code>	Design a function that prints every node's value in reverse, from tail to head, using prev pointers rather than restarting from head.

<code>int size();</code>	Design a function that returns the number of nodes currently in the list.
<code>bool isEmpty();</code>	Design a function that returns whether the list currently contains any nodes.
<code>~DoublyLinkedList();</code>	Design a destructor that deletes every remaining node so that no memory is leaked when the list is destroyed.

Complexity Requirement

- `insertAtHead` and `deleteAtHead` must run in $O(1)$.
- `insertAtTail` and `deleteAtTail` must run in $O(n)$ (no tail pointer for this task).
- `display`, `displayReverse`, and the destructor must run in $O(n)$.
- `size()` and `isEmpty()` must run in $O(1)$ if you maintain a running count, or $O(n)$ if you count nodes on demand. State which approach you used.

Required Behaviour

- Every insertion and deletion must correctly maintain both the next pointers and the prev pointers of any affected nodes, in both directions.
- Deleting from an empty list must not crash the program.
- Deleting the only remaining node must correctly reset head to `nullptr`.
- `display()` and `displayReverse()` on an empty list should each print a clear message such as "List is empty." rather than nothing at all.

Example

```
insertAtHead(20)
insertAtHead(10)
insertAtTail(30)
insertAtTail(40)

display()      -> 10 20 30 40
displayReverse() -> 40 30 20 10

deleteAtHead()
display()      -> 20 30 40

deleteAtTail()
display()      -> 20 30
```

Task 5: Palindrome Check (Doubly Linked List)

Using your `DoublyLinkedList<T>`, write a function that determines whether the list reads the same forwards and backwards.

```
bool isPalindrome();
```

Important Complexity Constraint

A solution that copies every value into an array and checks the array for a palindrome would work, but it does not take advantage of the doubly linked list's structure. You are required to check the list directly, using $O(n)$ time and $O(1)$ extra space, without building any auxiliary array.

Restrictions

- Do not copy values into an array or any other auxiliary structure.
- Do not use any STL container.
- Clearly state the time and space complexity of your solution in a comment.

Required Behaviour

- An empty list and a single-node list must both be treated as palindromes.

Example

```
List: 1 2 3 2 1
isPalindrome(): true

List: 1 2 3 4
isPalindrome(): false
```

Task 6: Delete a Given Node

Write a function that deletes a specific node from the list, given only a pointer to that node, without traversing the list from head to find it. You may assume the given node pointer always belongs to this list and is never nullptr.

```
void deleteNode(Node* nodeToDelete);
```

Important Complexity Constraint

A solution that searches for `nodeToDelete` by walking from head, as `deleteAtHead` and `deleteAtTail` effectively do, would take $O(n)$ time and misses the point of the task. You are required to remove the given node in $O(1)$ time, using only the information already reachable from the node pointer itself.

Restrictions

- Do not traverse the list from head to locate `nodeToDelete`.
- Do not use any STL container.
- Handle the case where the node being deleted is the current head correctly.
- Clearly state the time complexity of your solution in a comment.

Example

```
List: 100 200 300 400 500

// nodeToDelete points at the node containing 300
deleteNode(nodeToDelete)

display() -> 100 200 400 500
```

Submission Guidelines

Submit the following through Google Classroom:

1. Source code (.cpp file) for each completed task, i.e. 25I-XXXX_T1.cpp.
2. Screenshots showing successful execution of each task in a Word file named after your roll number.
3. Answers to the conceptual questions (If any) in the same Word document.
4. Late submissions will be penalized.

Important:

Programs that produce the correct output but violate the specified time-complexity requirement will not receive full credit for that task.